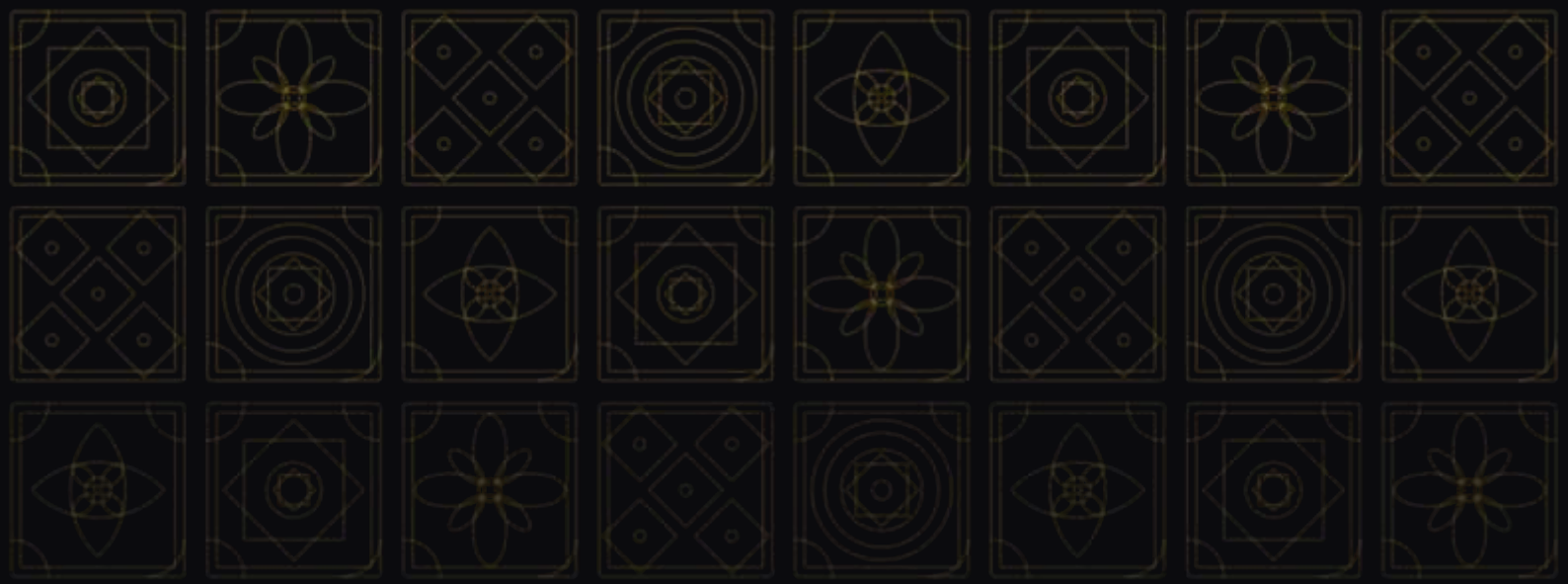




TRAINING PROGRAMME

DevSecOps

Build security into every stage of the delivery pipeline

57 HOURS

FOR DevOps engineers and developers who own a delivery pipeline



Mohamed Aziz BEN HAHHA

mohamedazizbenhaha.netlify.app

Overview

This programme adds security to a working DevOps pipeline, stage by stage, while keeping the pipeline fast. Participants model threats, scan code and dependencies, test the running application, then secure the software supply chain: hardened and scanned images, SBOMs, signatures, attestations and SLSA provenance checked before deployment. They scan infrastructure code, enforce policies as code, manage secrets with HashiCorp Vault, and harden and watch Kubernetes at runtime. The programme closes with a final project: a pipeline with a security gate at every stage and a security report.

At a glance

#	PHASE	FOCUS	HOURS
01	Foundations & Application Security	Shift-left, threat modelling, SAST, SCA and licences, DAST	12 h
02	Container & Supply Chain Security	Image scanning and hardening, SBOM, signing, SLSA provenance, admission	9 h
03	Infrastructure & Secrets	IaC scanning, policy as code, HashiCorp Vault, secret scanning	12 h
04	Kubernetes & Runtime Security	Pod Security Standards, RBAC hardening, NetworkPolicies, Falco	6 h
05	Final Project	A secure end-to-end pipeline and its security report	9 h
	Buffer	Catch-up, revision, extra lab time	9 h
	Total		57 h

Key facts

AUDIENCE	DevOps engineers and developers who own a delivery pipeline
PREREQUISITES	CI/CD, Terraform and Kubernetes delivery (course: DevOps Engineering, which builds on Docker & Kubernetes), or equivalent. Pipeline building is not taught again
DURATION	57 h: 48 h of chapters + 9 h buffer
FORMAT	Short lectures followed by guided labs; one application and pipeline carried through every phase
LAB ENVIRONMENT	Runs locally: one laptop per participant (16 GB RAM minimum, Docker, kind, VS Code); Jenkins and SonarQube on a server provided by the trainer; a public GitHub repository and GitHub Container Registry per team; Terraform code is scanned, not applied; no cloud account or cost
ASSESSMENT	A lab validated at the end of each phase and a final project with a security report presented to the group; certificate of completion for participants who complete both
OUTCOME	Participants can add and tune security gates for code, dependencies, the running application, images, infrastructure code, secrets and Kubernetes runtime, and report the residual risk

PHASE 01

12 H

Foundations & Application Security

Participants learn where security belongs in the pipeline and which gate stops what, model the threats of the course application with STRIDE, then add static analysis, dependency and licence checks, and dynamic scans of the running application. Validated lab: a pipeline that fails on a planted SQL injection, a vulnerable dependency and a ZAP finding, with the false positives triaged.

#	CHAPTER	CONTENT	FORMAT	HOURS
1.1	DevSecOps Principles	Shift-left, threat modelling with STRIDE, security gates and their thresholds	Lecture + Workshop	3 h
1.2	Static Analysis (SAST)	SonarQube security rules, Semgrep rules for injection flaws, custom rules, triaging false positives	Lecture + Lab	3 h
1.3	Dependencies & Licences (SCA)	OWASP Dependency-Check, vulnerable dependency updates, licence compliance	Lecture + Lab	3 h
1.4	Dynamic Testing (DAST)	ZAP baseline, full and API scans in the pipeline	Lecture + Lab	3 h

PHASE 02

9 H

Container & Supply Chain Security

Participants harden and scan the course images, then prove where they come from: an SBOM for every image, signatures and attestations with Cosign and Sigstore, SLSA provenance from the build, and admission policies that refuse unsigned images in the cluster. Validated lab: an unsigned image rejected by the cluster while the signed, attested build is admitted.

#	CHAPTER	CONTENT	FORMAT	HOURS
2.1	Image Scanning & Hardening	Trivy, minimal and distroless base images, non-root images, fixing findings	Lecture + Lab	3 h
2.2	SBOM & Signing	SBOM with Syft, signing with Cosign and Sigstore, keyless signing	Lecture + Lab	3 h
2.3	Attestations & Admission	Attestations, SLSA provenance, admission of signed images with the Sigstore policy-controller	Lecture + Lab	3 h

PHASE 03

12 H

Infrastructure & Secrets

Participants scan the course Terraform and Kubernetes code with Checkov and Trivy, enforce rules in the cluster with OPA Gatekeeper or Kyverno, then remove secrets from code: HashiCorp Vault with dynamic secrets, its Jenkins and Kubernetes integrations, and secret scanning with gitleaks. Validated lab: a pipeline that blocks a public bucket in Terraform and a committed password, and an application that reads short-lived database credentials from Vault.

#	CHAPTER	CONTENT	FORMAT	HOURS
3.1	laC Security	Checkov, Trivy IaC scanning (formerly tfsec), fixing misconfigurations in Terraform and Kubernetes manifests	Lecture + Lab	3 h
3.2	Policy as Code	Kyverno in the lab, compared with OPA Gatekeeper and the built-in ValidatingAdmissionPolicy; writing and testing policies, audit and enforce modes, image verification with Kyverno verifyImages	Lecture + Lab	3 h
3.3	HashiCorp Vault	Architecture, secrets engines, dynamic secrets, policies and authentication	Lecture + Lab	3 h
3.4	Secrets in the Pipeline	Vault integration with Jenkins and Kubernetes, secret scanning with gitleaks, pinning third-party actions by commit SHA	Lecture + Lab	3 h

PHASE 04

6 H

Kubernetes & Runtime Security

Participants harden the cluster that runs the course application with Pod Security Standards, least-privilege RBAC and default-deny NetworkPolicies, then detect attacks while they happen with Falco rules. Validated lab: a shell opened in a running container detected by Falco and an alert raised.

#	CHAPTER	CONTENT	FORMAT	HOURS
4.1	Kubernetes Hardening	Pod Security Standards, RBAC hardening, NetworkPolicies	Lecture + Lab	3 h
4.2	Runtime Security	Falco rules, runtime detection and alerting	Lecture + Lab	3 h

PHASE 05

9 H

Final Project

Teams of two or three secure the delivery pipeline of a provided application, reusing the phase labs, then present the result and its residual risks as they would to a security officer.

#	CHAPTER	CONTENT	FORMAT	HOURS
5.1	Brief & Threat Model	Read the brief, model the threats, choose the gates and their thresholds	Project	3 h
5.2	Build the Secure Pipeline	SAST, SCA, DAST, image scanning and signing, IaC scanning, secrets from Vault	Project	3 h
5.3	Runtime & Report	Cluster policies and Falco rules; security report with findings, fixes and accepted risks	Project	1.5 h
5.4	Demo & Review	Live demo of a vulnerable change stopped by the pipeline; questions and feedback	Project	1.5 h

FINAL PROJECT

A Secure Delivery Pipeline

A pipeline with a security gate at every stage: code, dependencies, running application, images, infrastructure code, secrets and Kubernetes runtime, with signed images admitted to the cluster and a security report that states what is fixed and what risk remains.

STACK GitHub Actions or Jenkins, SonarQube, Semgrep, OWASP Dependency-Check, ZAP, Trivy, Syft, Cosign, Checkov, Kyverno or OPA Gatekeeper, HashiCorp Vault, gitleaks, Falco, Kubernetes

DELIVERABLE Repository with pipeline and policies, threat model, security report, live demo